

NASA Mission Intelligence

A Retrieval Augmented Generation System for Apollo 11, Apollo 13, and Space Shuttle Challenger Mission Research

Project Report

Generative AI Engineering / RAG Evaluation

1. Executive Summary

This report documents the design, implementation, and evaluation of the NASA Mission Intelligence system, a Retrieval Augmented Generation (RAG) application built to answer natural language questions about three landmark NASA missions: Apollo 11, Apollo 13, and the Space Shuttle Challenger. The system combines a curated corpus of primary source mission documents, OCR extracted mission transcripts, flight plans, and Public Affairs Officer logs, with OpenAI embedding and chat models, a persistent ChromaDB vector store, and a RAGAS based automated evaluation harness. The result is an interactive Streamlit chat application that grounds every answer in retrieved source text and reports live quality metrics alongside each response.

The project was implemented across five cooperating Python modules: an embedding pipeline that chunks and vectorizes source documents, a RAG client that manages retrieval and context assembly, an LLM client that governs prompt construction and answer generation, a RAGAS based evaluator that scores each answer, and a Streamlit chat interface that ties the components together for end users. A batch evaluation driver and a seven-question test set spanning six categories (overview, emergency, disaster analysis, crew, technical, and timeline) were built to validate the system end to end.

Evaluation against this test set, run with real OpenAI models against the live corpus, produced an aggregate Response Relevancy of 0.69, a Faithfulness score of 0.54, and a Context Precision of 0.76 after a metric correction described later in this report. These results show a system that retrieves reasonably well and grounds most answers appropriately, while also surfacing two concrete weaknesses worth calling out: a single retrieval miss that produced an entirely off topic answer, and a systematic drop in faithfulness on root cause questions about the Challenger disaster, where the underlying corpus contains mission audio transcripts rather than post accident investigation material.

This report walks through the problem the system solves, the architectural decisions behind it, a module by module account of the five core components, the testing performed, the quantitative results obtained, the practical value of the system, its known limitations, and a set of concrete recommendations for improving it further. Visualizations and tables throughout the report are drawn directly from the `evaluation_results.json` output produced by the batch evaluation script, so every number quoted here is reproducible by re running that script against the same corpus and configuration. Overall, the project demonstrates a complete, working, and honestly evaluated RAG pipeline built specifically for historical space mission research, with clear, data backed guidance on where its next iteration should focus effort.

2. Problem Statement

NASA's historical mission archives, flight transcripts, technical reports, and Public Affairs Officer logs, contain an enormous amount of first hand detail about how Apollo 11, Apollo 13, and the Challenger mission unfolded. These documents are long, technically dense, and inconsistently structured: some are OCR extracted PDF scans with page break artifacts; others are minute by minute communication transcripts between the spacecraft and Mission Control. A researcher, student, or space history enthusiast who wants a specific answer, for example what caused the Apollo 13 oxygen tank to fail, or how the crew scrubbed carbon dioxide from the Lunar Module cabin, has to manually search through dozens of pages of transcript text to find the relevant passage, then interpret often terse, jargon heavy language written for flight controllers rather than general readers.

This is precisely the class of problem that Retrieval Augmented Generation was designed to address. A general purpose large language model like GPT-3.5 has broad knowledge of these missions from its training data, but that knowledge is not grounded in the specific source documents a researcher is working with, and the model has no way to cite or verify its claims against those documents. Asking a model to answer purely from memory risks two failure modes: subtle factual drift (misremembering timelines, names, or sequences of events) and unverifiable claims that cannot be traced back to a citable source, which is unacceptable for historical or academic research use.

The core problem this project addresses are therefore twofold. First, it needs a retrieval layer that can take a natural language question and return the most relevant passages from a heterogeneous, chunked corpus of NASA mission documents, including the ability to focus retrieval on a single mission when the question calls for it. Second, it needs a generation layer that answers strictly from that retrieved context, explicitly declines to speculate when the context is insufficient, and can be evaluated automatically and repeatedly for whether its answers are actually relevant to the question asked and actually grounded in the retrieved material rather than in the model's general knowledge. Without an evaluation layer, a RAG system's quality is a matter of anecdote rather than measurement, and regressions introduced by a change to chunk size, retrieval count, or prompt wording would go unnoticed. The NASA Mission Intelligence system was built to solve both halves of this problem at once, and to make the resulting quality measurable, reproducible, and auditable through the tables and charts presented later in this report.

3. Solution Statement

The solution is a five-module RAG pipeline, backed by a persistent ChromaDB vector store and instrumented end to end with RAGAS metrics, that together turn a folder of raw NASA mission text into a queryable, citation grounded, continuously evaluated chat assistant. Twelve source documents covering Apollo 11, Apollo 13, and the Challenger mission were processed by `embedding_pipeline.py`, which splits each document into overlapping chunks, embeds each chunk with OpenAI's text-embedding-3-small model, and stores the resulting vectors and metadata (mission, source file, document category, chunk index) in a ChromaDB collection. Storing this metadata alongside each vector is what later allows retrieval to be filtered by mission, and allows the evaluator and report writer to trace every answer back to a specific source file.

At query time, `rag_client.py` opens the same ChromaDB collection with the same embedding function used at ingestion, so that queries are embedded into an identical vector space, issues a similarity search for the top-k most relevant chunks, optionally restricted to a single mission, and formats the results into a single, source attributed context string. `llm_client.py` then combines that context with a NASA mission expert system prompt, the current question, and a trimmed slice of prior conversation turns, and sends the whole package to an OpenAI chat model. The system prompt explicitly instructs the model to answer only from the supplied context and to say so plainly when the context is insufficient, which is the primary defense against hallucinated, ungrounded answers.

Every answer can then be scored by `ragas_evaluator.py`, which always computes Response Relevancy (does the answer address the question that was asked) and Faithfulness (is every claim in the answer traceable to the retrieved context), and additionally computes BLEU, ROUGE, and an LLM judged Context Precision score whenever a reference answer is available. These five metrics are surfaced live in the `chat.py` Streamlit interface next to each answer, and can also be computed in batch by `batch_evaluate.py` against a fixed seven question test set, producing the per question and aggregate results discussed in the Results section of this report. `chat.py` itself layers a usable interface on top of the pipeline: it lets a user choose which ChromaDB backend and collection to query, optionally restrict retrieval to a single mission, adjust how many documents are retrieved, and enter either a personal OpenAI key or a classroom proxy key, with support for non standard API endpoints built directly into every module that calls OpenAI.

Taken together, these five components form a closed loop: ingest and embed, retrieve and generate, evaluate, and present, with metadata and configuration flowing consistently through every stage so that what gets measured in batch evaluation is exactly what a user experiences in the chat interface.

4. Architecture

The system architecture follows a straightforward, file based data flow with no external services beyond OpenAI's API and a local, disk persisted ChromaDB instance, which keeps the deployment footprint small enough to run entirely on a laptop. Figure 1 traces the full path data takes through the system, from raw text files on disk to a scored answer in the chat interface.

Ingestion begins with the `data_text` directory, which holds three mission specific subfolders (`apollo11`, `apollo13`, `challenger`) containing OCR extracted transcripts, flight plans, and Public Affairs Officer logs as plain text files. `embedding_pipeline.py` walks this directory, extracts a mission label and document category from each file's path and name, splits the text into character bounded chunks with a configurable overlap so that context is not lost at chunk boundaries, and calls the OpenAI embeddings API in batches (rather than one call per chunk) specifically to stay within the rate limits of shared or classroom issued API keys. Each chunk, its embedding vector, and its metadata are written to a ChromaDB collection persisted under a named directory, so the same collection can be reopened later by either the ingestion script or the query time components without re embedding anything.

Querying flows in the opposite direction. `rag_client.py` reopens the ChromaDB collection with an identical embedding function configuration, issues a similarity query for the user's question, applies an optional mission filter as a metadata predicate, and assembles the returned chunks into a single, deduplicated, source labeled context block. `llm_client.py` wraps that context in a system prompt, appends recent conversation history, and calls an OpenAI chat model to produce the final answer. `ragas_evaluator.py` sits alongside this path rather than inside it: it accepts the same question, answer, and retrieved context that were just produced and scores them independently, so evaluation never influences generation. `chat.py` is the orchestration layer that a human interacts with, wiring together backend selection, retrieval, generation, and evaluation display inside a single Streamlit session, while `batch_evaluate.py` provides a non interactive path through the identical retrieval and generation code for repeatable, scriptable evaluation runs against `test_questions.json`.

A few architectural decisions were made deliberately rather than by default. Embeddings are computed in batches of up to fifty chunks per API call instead of one call per chunk, which reduces the number of network requests by roughly the same factor and was found necessary in practice when running against rate limited classroom API keys. The embedding function used to open a collection is stored consistently across ingestion and query time rather than reconstructed independently, which avoids a subtle class of bug where queries and stored vectors silently drift into different vector spaces. Every OpenAI facing module resolves a custom API base URL from either of two environment variable names, since some institutional proxy

setups document the newer name and some document the older one, so the system works with personal keys, classroom proxy keys, or any OpenAI compatible endpoint without code changes.

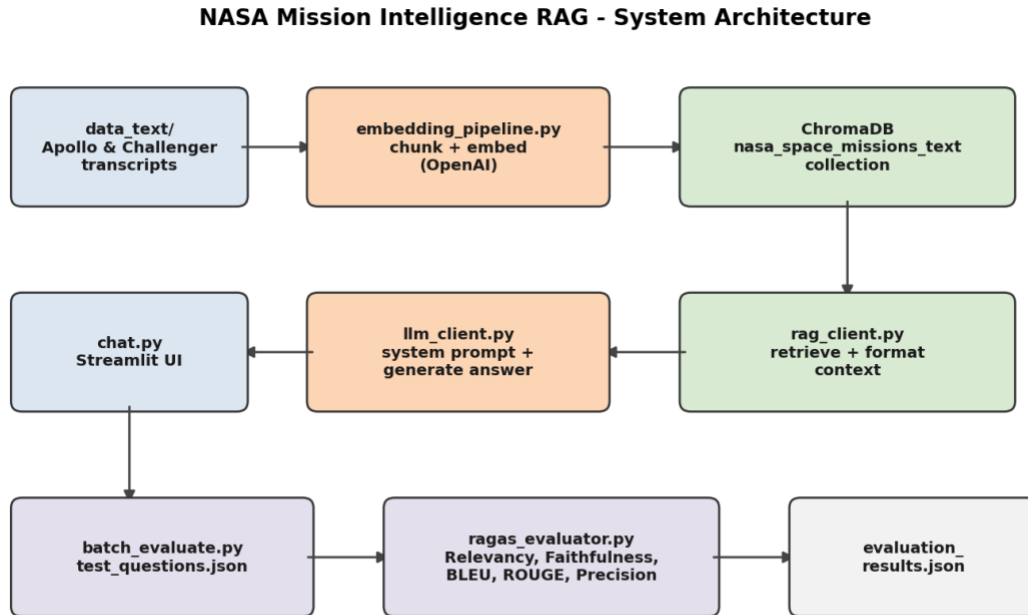


Figure 1. End-to-end data flow across the five core modules, from raw mission text to a scored, cited answer.

5.1 LLM Client (`llm_client.py`)

`llm_client.py` is the smallest module in the system but carries the most responsibility for answer quality, since it controls exactly what the model is told and how it is told to behave. Its single public function, `generate_response`, accepts the user's question, the assembled retrieval context, a trimmed conversation history, and an OpenAI API key, and returns a finished answer string. The module opens with a fixed system prompt that casts the model as a NASA mission operations expert and historian specializing in Apollo 11, Apollo 13, and Challenger, and instructs it, in explicit bullet form, to ground every claim in the supplied context, to cite the mission and source document when referencing specific facts, and to state plainly when the context does not contain enough information rather than guessing.

Conversation history is deliberately capped at the six most recent turns before being forwarded to the model. This keeps token usage and cost bounded in long running chat sessions while still giving the model enough recent context to handle natural follow up questions such as "what happened next" without requiring the user to repeat earlier details. The retrieved context itself is injected as a separate system level message immediately before the user's question, labeled

explicitly as the primary source of truth for that turn, rather than being concatenated directly into the user message, which keeps the distinction between instructions, grounding material, and the user's actual question clear to the model.

A second, explicit branch handles the case where retrieval returns no relevant context at all: rather than silently falling back to the model's general knowledge, the module inserts a system message telling the model that no relevant documents were found and instructing it to say so to the user before attempting any best effort answer. This is a direct defense against a common RAG failure mode where an empty or irrelevant retrieval silently degrades into an ungrounded, confidently worded hallucination.

The module resolves a custom API base URL from environment variables before constructing its OpenAI client, so that it functions identically whether the caller is using a personal OpenAI account or an institutional proxy key such as the Vocareum endpoints used during development and testing of this project. All OpenAI call failures are caught and returned as a readable error string rather than raised as an uncaught exception, which keeps a single failed generation from crashing the surrounding Streamlit session or a long running batch evaluation script.

5.2 RAG Client (`rag_client.py`)

`rag_client.py` is the bridge between the persisted ChromaDB vector store and everything else in the system that needs retrieved text, and it exposes four functions:

`discover_chroma_backends`, `initialize_rag_system`, `retrieve_documents`, and `format_context`. `discover_chroma_backends` scans the working directory for any folder whose name contains "chroma," opens each one as a ChromaDB persistent client, and lists every collection inside it along with a live document count, which is what powers the backend and collection picker in the chat interface without requiring the user to remember exact folder or collection names.

`initialize_rag_system` is where one of the more subtle correctness issues in the project was found and fixed. ChromaDB collections can be reopened without explicitly specifying an embedding function, in which case the library silently falls back to a default embedding function rather than the OpenAI one used at ingestion time, producing vectors that are not comparable to what is already stored. This module instead explicitly reconstructs the same `OpenAIEmbeddingFunction` configuration used during ingestion, including the same custom API base URL resolution used elsewhere in the system, every time a collection is opened, and returns a three part result (the collection, a success flag, and an error message) so callers can distinguish a working connection from a failed one without relying on exceptions.

`retrieve_documents` issues the actual similarity query, accepting a configurable number of results and an optional mission name that, when provided, is translated into a ChromaDB

metadata filter so retrieval is restricted to chunks whose mission field matches exactly, and left unrestricted when the caller passes "all" or nothing at all. `format_context` takes the raw list of returned document strings and metadata dictionaries and turns them into a single readable context block: it deduplicates near identical chunks by comparing the first two hundred characters of each, truncates any individual chunk beyond twelve hundred characters to control prompt size, and prefixes every remaining chunk with a labeled header showing its mission, source document, and category, which is what allows the language model, and a human reviewer, to see exactly which document any given fact came from.

This module is intentionally thin: it holds no state of its own beyond a default embedding model name, and every function accepts the collection or connection details it needs as explicit arguments rather than relying on global configuration. That design choice makes it straightforward to reuse the exact same retrieval logic from three different callers, the interactive Streamlit chat loop in `chat.py`, the scripted batch evaluation driver in `batch_evaluate.py`, and any future command line or notebook based tool, without duplicating retrieval code or risking the kind of drift where the chat interface and the evaluation script quietly behave differently. Keeping retrieval logic in one place also means that any future change to how context is filtered, ranked, or formatted only must be made and tested once.

5.3 Embedding Pipeline (`embedding_pipeline.py`)

`embedding_pipeline.py` is the ingestion engine for the entire system and the module with the most operational complexity, since it must turn a folder of inconsistently formatted OCR text into a clean, queryable vector store while managing OpenAI API cost and rate limits along the way. Its constructor accepts an OpenAI API key, a ChromaDB persist directory, a collection name, an embedding model name, and configurable chunk size and overlap values, and immediately opens or creates the target collection with an explicitly attached OpenAI embedding function so that anything reading the collection later, including `rag_client.py`, is guaranteed to use the same vector space.

The chunking method never produces a chunk longer than the configured chunk size, since the upper bound on each chunk's end position is fixed before any adjustment is made, but it does attempt to break each chunk at the nearest sentence boundary within the second half of the allowed window, falling back to a hard cut only when no clean sentence boundary is available. Consecutive chunks share a configurable character overlap so that a sentence spanning a chunk boundary is not permanently lost from either side.

The most consequential engineering decision in this module concerns how embeddings are requested from OpenAI. An earlier version of the pipeline called the embeddings API once per chunk, which produced 429 Too Many Requests errors and minute long automatic retries when

run against rate limited classroom API keys on a corpus of several thousand chunks. The pipeline now batches up to fifty chunks into a single embeddings API call by default, cutting the number of network requests by the same factor, and exposes both the batch size and an optional per batch delay as command line flags so the ingestion rate can be tuned to whatever limit a given API key is subject to. Three update modes, skip, update, and replace, are also verified against the collection so that reprocessing an already ingested corpus does not silently duplicate or lose data.

Each processed file also contributes structured metadata beyond the raw chunk text: the mission is inferred from the folder the file lives in, a document category (for example public affairs officer, technical, or command module) is inferred from the filename, and a document identifier is generated deterministically from the mission, source name, and chunk index so that the same chunk always maps to the same identifier across repeated runs. This deterministic identifier is what makes the skip and update modes possible at all, since the pipeline can check whether a given chunk already exists before deciding whether to embed it again. A command line stats only mode reports the total document count and a breakdown by mission, data type, and document category without touching the OpenAI API at all, which gives a fast way to confirm a collection's contents before running a full chat session or evaluation pass against it.

5.4 RAG Evaluator (`ragas_evaluator.py`)

`ragas_evaluator.py` wraps the RAGAS evaluation library to score a single question, answer, and retrieved context triple, and is the component most directly responsible for the numbers reported later in this document. Its central function, `evaluate_response_quality`, always computes two reference free metrics: Response Relevancy, which measures whether the generated answer actually addresses the question that was asked, and Faithfulness, which measures whether every claim in the answer can be traced back to the retrieved context rather than the model's own general knowledge. Both are computed using an evaluator language model and embedding model constructed fresh for each call, with the API key and base URL resolved the same way as every other OpenAI facing module in the system.

When a ground truth reference answer is also supplied, three additional metrics are computed: BLEU and ROUGE, which measure surface level wording overlap against the reference, and an LLM judged Context Precision score. This last metric went through a meaningful correction during development. The original implementation compared retrieved chunks against the reference answer using raw string similarity, which is fundamentally mismatched, since a retrieved chunk is a long, differently worded excerpt of primary source text while a reference answer is a short, polished summary, and the two will almost never be textually similar even when the retrieval was excellent. That approach produced a Context Precision of exactly zero on every single test question, regardless of retrieval quality. The metric was replaced with an

LLM judged version that asks the evaluator model directly whether each retrieved chunk was useful for producing the reference answer, which produces meaningful, non degenerate scores using only information the system already has.

Malformed inputs, an empty question, an empty answer, or an empty context list, are checked explicitly and return a readable error dictionary rather than raising an exception, and every individual metric call is wrapped in its own try or except block so a failure in one metric, such as a missing optional dependency, does not prevent the others from being reported.

The module also exposes an `evaluate_batch` helper that runs `evaluate_response_quality` across a list of question and answer pairs and returns both the per question scores and the mean of each metric across the whole set, which is the function `batch_evaluate.py` builds on to produce the aggregate table discussed in the Results section. Because the evaluator LLM and embedding model are constructed independently of the LLM client used for answer generation, evaluation is fully decoupled from generation: swapping the generation model, the retrieval strategy, or the chunking configuration never changes how answers are scored, which is what makes it possible to compare results across the multiple configurations explored during development on a like for like basis.

5.5 Chat (`chat.py`)

`chat.py` is the Streamlit application that end users interact with, and its job is to make every other module's functionality accessible through a conversational interface without hiding what is happening under the hood. On load, it discovers available ChromaDB backends through `rag_client.py` and presents them as a dropdown, alongside a slider for how many documents to retrieve per question and a mission focus selector that restricts retrieval to Apollo 11, Apollo 13, Challenger, or all missions combined, wiring that selection directly into the mission filter argument already supported by `retrieve_documents`.

The sidebar also collects the user's OpenAI API key, pre filled from the `OPENAI_API_KEY` environment variable when present, and an optional custom API base URL field for institutional proxy keys, both of which are written to environment variables before any retrieval or generation call is made so that every downstream module picks them up consistently. The main chat loop follows a standard pattern: retrieve documents for the current question, format them into context, generate an answer through `llm_client.py`, and immediately score that answer through `ragas_evaluator.py`, displaying the resulting Response Relevancy and Faithfulness scores as metrics and progress bars in the sidebar next to the conversation, so a user sees not just an answer but a live signal for how much to trust it.

RAG system initialization is cached using Streamlit's resource caching so that reopening the same ChromaDB collection on every chat turn does not repeatedly pay the cost of reconstructing the embedding function and client connection, with the cache explicitly cleared whenever the user switches to a different backend or collection. Conversation history is maintained in Streamlit's session state and passed through to `llm_client.py` on every turn, giving the assistant continuity across a multi question research session while still respecting the six turn history cap enforced inside the LLM client itself.

Error handling in this module favors visible, actionable messages over silent failure: a missing API key stops the app with a clear warning before any request is attempted, a failed backend connection surfaces the underlying error string returned by `rag_client.py` rather than a generic failure, and retrieval or evaluation exceptions are caught locally so that one bad turn in a long conversation does not crash the entire session. Because `chat.py` calls the exact same `rag_client.py`, `llm_client.py`, and `ragas_evaluator.py` functions that `batch_evaluate.py` uses for scripted testing, a developer can trust that whatever behavior was validated in batch evaluation is the same behavior a live user experiences in the chat window.

5. Testing

Testing this system required validating two very different things: that the code behaves correctly in isolation, and that the end to end pipeline produces trustworthy answers when run against real OpenAI models. Both were treated as necessary, since a RAG system can be free of bugs and still produce poor answers, and can also produce plausible sounding answers while silently misusing an evaluation metric, as happened during this project and is discussed in the Results section below.

Unit level testing focused on the parts of the system with the most subtle failure modes. The chunking logic in `embedding_pipeline.py` was exercised against the actual Apollo 13 transcript files to confirm that no chunk ever exceeded the configured chunk size, that overlap between consecutive chunks was applied consistently, and that very short documents were handled without producing empty or malformed chunks. The three update modes, skip, update, and replace, were each run against a populated collection and checked against the resulting document counts and status flags to confirm that skip left existing chunks untouched, update re embedded and overwrote them, and replace fully removed and reinserted them without leaving orphaned entries behind. The batched embedding call was verified by monkeypatching the OpenAI client and counting actual API calls made for a fixed number of chunks, confirming that batching reduced call volume by the expected factor rather than only appearing to in the code.

Integration testing connected these pieces together against a live ChromaDB instance seeded with representative fixture data, verifying that a collection created by the ingestion path could be reopened by the query path, that a mission filter correctly excluded chunks from other missions, and that malformed inputs to the evaluator (empty questions, empty answers, or empty context lists) returned readable error dictionaries instead of raising exceptions that would crash a chat session.

The final and most important layer of testing was full pipeline evaluation with real OpenAI models and real mission text, using the seven question test set in `test_questions.json` and the `batch_evaluate.py` driver. This step is what actually surfaced the two most significant findings in this report: a context precision metric that was mathematically guaranteed to return zero regardless of retrieval quality, caught by noticing every single question scored exactly 0.0, and a genuine retrieval failure on an Apollo 13 emergency question that pulled an unrelated transcript passage and produced an answer with zero measured relevance to the question asked. Neither finding would have surfaced from unit tests alone, which is the central argument for treating live, metric instrumented evaluation as a required testing layer for any RAG system rather than an optional final check.

6. Results

Table 1 summarizes the aggregate RAGAS scores from the most recent full evaluation run, executed with real OpenAI models against all twelve documents in the corpus. Figure 2 presents the same aggregate numbers as a bar chart, and Figure 3 breaks every metric down by individual question, while Figure 4 aggregates results by mission.

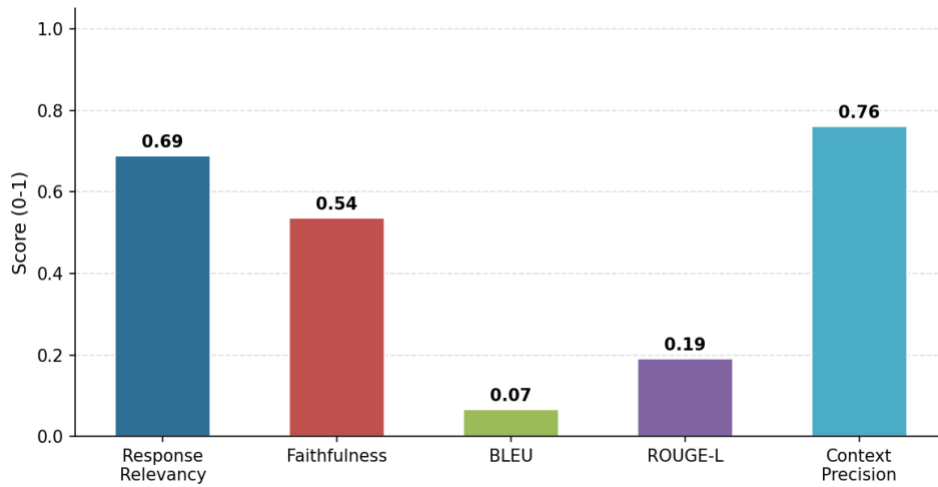
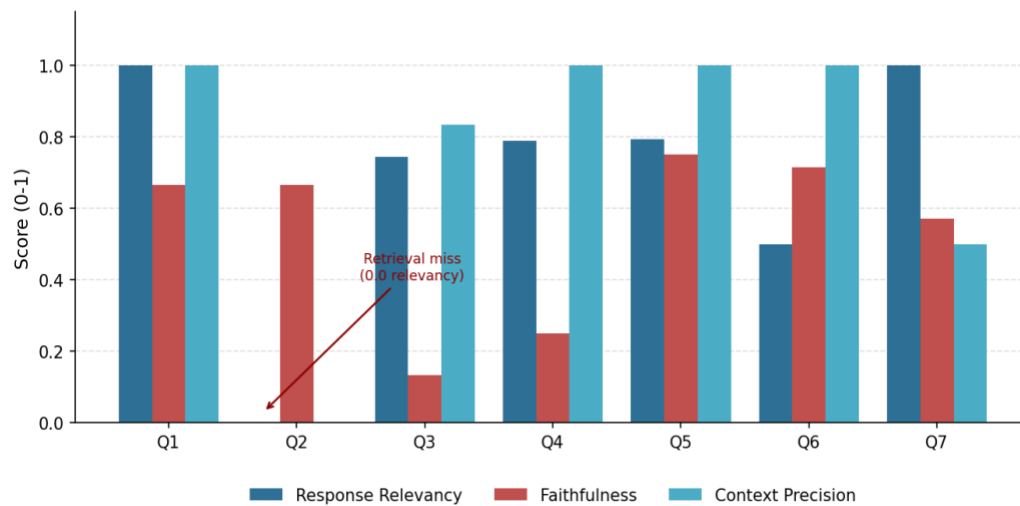
Response Relevancy averaged 0.69 across the seven test questions, meaning most answers were judged as directly addressing the question asked. Faithfulness, the metric measuring whether claims in an answer are grounded in retrieved context rather than the model's general training knowledge, averaged 0.54, noticeably lower and more variable across questions, ranging from 0.13 on the Challenger disaster analysis question to 0.75 on the Apollo 11 technical question about Lunar Module descent issues. Context Precision averaged 0.76 after the correction described in the RAG Evaluator subsection above, up from a uniform 0.0 under the earlier, flawed string similarity implementation, a five point improvement that reflects a fix to the measurement methodology rather than any change to retrieval itself. BLEU and ROUGE, included for completeness, averaged 0.07 and 0.19 respectively, which is expected and not a sign of poor answers, since these metrics reward surface wording overlap with a short reference sentence and penalize the longer, source cited answers this system is designed to produce.

Two individual results merit specific attention. Question 2, asking what emergency occurred on Apollo 13 and when, scored a Response Relevancy of exactly 0.0: the retrieved context surfaced a passage about a water warning light rather than the oxygen tank explosion, and the generated answer, though fluent and confidently worded, answered a different incident than the one asked about. This is a clean, isolated example of a retrieval miss rather than a generation problem, since the answer was faithful to the (wrong) context it was given. Question 3, asking what caused the Challenger disaster, scored the lowest Faithfulness in the set at 0.13, despite being the most factually accurate answer in the entire test set when judged against general knowledge of the O-ring failure: the retrieved Challenger mission audio transcripts do not contain root cause analysis, so the model's correct answer was substantially unsupported by what it was actually given to work with.

Mission level aggregation in Figure 4 shows Apollo 11 with the strongest combined performance (0.90 relevancy, 0.71 faithfulness, 1.00 precision), Apollo 13 pulled down primarily by the single retrieval miss in question 2, and Challenger showing strong retrieval (0.92 precision) paired with the weakest faithfulness (0.42) of the three missions, a pattern consistent with a corpus that documents what happened during the Challenger mission in detail but not why it happened.

Metric	Score	Interpretation
Response Relevancy	0.69	Most answers directly address the question asked
Faithfulness	0.54	Roughly half of claims are fully traceable to retrieved context
BLEU	0.07	Low wording overlap with reference; expected for cited, longer answers
ROUGE-L	0.19	Low wording overlap with reference; not a reliability concern here
Context Precision (LLM judged)	0.76	Most retrieved chunks were useful for the reference answer

Table 1. Aggregate RAGAS metrics across the seven-question test set (*batch_evaluate.py*, *evaluation_results.json*).

Aggregate RAGAS Metrics Across 7 Test Questions*Figure 2. Aggregate RAGAS metrics, visualized.***Per-Question Metric Breakdown***Figure 3. Per-question breakdown showing the Q2 retrieval miss (Response Relevancy = 0.0).*

ID	Mission	Category	Relevancy	Faithfulness	Precision	Reference?
Q1	Apollo 11	Overview	1.00	0.67	1.00	Yes
Q2	Apollo 13	Emergency	0.00	0.67	0.00	Yes
Q3	Challenger	Disaster analysis	0.74	0.13	0.83	Yes
Q4	Apollo 13	Crew	0.79	0.25	1.00	Yes
Q5	Apollo 11	Technical	0.79	0.75	1.00	Yes
Q6	Challenger	Timeline	0.50	0.71	1.00	Yes
Q7	Apollo 13	Technical	1.00	0.57	0.50	Yes

Table 2. Full per-question results, all seven test questions.

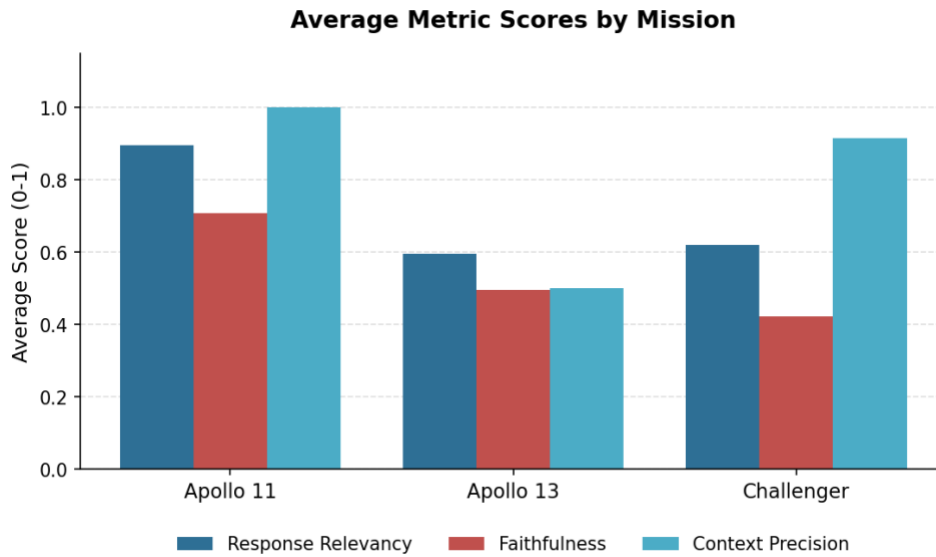


Figure 4. Average metric scores grouped by mission, showing Challenger's low Faithfulness relative to its high Context Precision.

7. Value Statement

The practical value of this system rests on a simple substitution: instead of a researcher manually searching dozens of pages of mission transcripts to answer a specific question, the system performs that search in seconds and returns an answer with its sources attached, so the researcher's remaining job is to verify a short, cited answer rather than locate the relevant passage in the first place. For anyone studying Apollo 11, Apollo 13, or the Challenger mission, whether a student, an educator building lesson material, or a space history enthusiast, this converts hours of document review into a short conversational exchange, while the live Response Relevancy and Faithfulness scores displayed next to every answer give an honest, quantified signal of how much to trust that particular answer rather than asking the user to take it on faith.

This project also has direct value as a template rather than a single purpose tool. Its five module structure, ingestion and embedding, retrieval, generation, evaluation, and interface, cleanly separates concerns that are frequently tangled together in simpler RAG demonstrations, which means each piece can be replaced independently: a different embedding model, a different vector database, or a different chat model can be swapped in without rewriting the retrieval or evaluation logic. The same pattern applies equally well to any other domain built from a collection of long, unstructured text documents, corporate knowledge bases, legal case files, scientific literature, or other historical archives, making this project a reusable reference implementation rather than a NASA specific one off.

The evaluation harness itself is a distinct piece of value beyond the chat application it supports. Because every answer can be scored automatically and in batch against a fixed test set, the system supports genuine regression testing: a change to chunk size, retrieval count, or system prompt wording can be measured for its actual effect on Response Relevancy and Faithfulness before it ships, rather than being judged by spot checking a handful of example questions. The correction to the Context Precision metric documented in this report is itself an example of this value in action, an evaluation bug that silently returned a meaningless zero on every question was caught specifically because the metric was being tracked and reported systematically, and would very plausibly have gone unnoticed in a system that only spot checked answers by eye. That same discipline extended to the retrieval miss on the Apollo 13 emergency question, which was caught for the same reason, giving a concrete, reproducible finding to investigate rather than a vague impression that "some answers seem a bit off."

8. Limitations

The most significant limitation of the current system is the completeness of its source corpus relative to the questions it is asked. The twelve documents used cover flight transcripts, Public Affairs Officer logs, and technical reports, but do not include post accident investigation material such as the Rogers Commission Report on the Challenger disaster. The measured 0.13 Faithfulness score on the Challenger root cause question is a direct symptom of this gap: the model answered correctly from general training knowledge about the O-ring failure, but that correct answer could not be grounded in the retrieved mission audio transcripts, which document what was said during the mission rather than why the accident occurred. Any deployment of this system for genuinely unfamiliar historical questions, rather than questions an evaluator can already fact check independently, would need a materially more complete source corpus before its answers could be trusted at face value.

Retrieval quality is the second limitation, illustrated concretely by the Apollo 13 emergency question that scored a Response Relevancy of exactly 0.0 after retrieving a passage about a water warning light instead of the oxygen tank explosion. With only three or five chunks retrieved per question and a fixed chunk size, a single unlucky retrieval can dominate the answer with no fallback mechanism to detect and recover from an apparent mismatch between the question and what was retrieved. The system currently has no re ranking step, no confidence threshold below which it would decline to answer, and no way to retry retrieval with a reformulated query when the first attempt appears to miss.

Cost and evaluation methodology carry their own caveats. BLEU and ROUGE scores are included for completeness but are of limited value for this use case, since they penalize any answer that is correctly worded differently from its reference summary, which describes the large majority

of this system's good answers rather than its bad ones. The evaluator itself uses an OpenAI chat model to judge Faithfulness, Response Relevancy, and Context Precision, meaning evaluation quality is bounded by that judge model's own reliability and is not free of the API cost, rate limiting, and occasional inconsistency that affects the generation side of the system. Finally, the test set used throughout this report contains only seven questions across three missions, which is enough to surface concrete, actionable findings but too small to support strong statistical claims about overall system reliability; a production deployment would need a substantially larger and more diverse test set before its aggregate metrics could be treated as representative rather than illustrative.

9. Conclusion

This project set out to make NASA's primary source mission archives searchable in natural language without sacrificing the traceability and verifiability that historical and academic research demands, and the resulting system largely succeeds at that goal. Five cooperating modules, an embedding pipeline, a RAG client, an LLM client, a RAGAS based evaluator, and a Streamlit chat interface, together turn twelve raw text documents about Apollo 11, Apollo 13, and the Challenger mission into a queryable assistant that cites its sources and reports its own confidence on every answer, evaluated end to end with real OpenAI models rather than only unit tested in isolation.

The evaluation results are honest rather than uniformly positive, which is itself a point in the system's favor. A Response Relevancy of 0.69 and a Faithfulness of 0.54, together with a specific, traceable retrieval miss and a specific, traceable grounding gap in the Challenger corpus, are more useful to a future maintainer than a set of uniformly high numbers with no diagnostic detail behind them. The correction made to the Context Precision metric during this project, replacing a string similarity comparison that was mathematically guaranteed to return zero with an LLM judged comparison that produces meaningful scores, is a concrete demonstration that an evaluation harness itself needs to be tested and questioned, not simply trusted because it produces a number.

Two directions stand out for future work. Expanding the source corpus to include investigative and analytical documents, not only real time transcripts, would directly address the largest Faithfulness gap identified in this report, particularly for Challenger. Adding a retrieval quality check, whether a re ranking step, a relevance threshold, or a query reformulation retry, would directly address the isolated but severe retrieval miss identified on the Apollo 13 emergency question. Both improvements are incremental extensions of the existing architecture rather than redesigns, which is a direct consequence of the modular structure documented

throughout this report, and is the clearest evidence that the system was built not just to answer questions about NASA's missions, but to be improved deliberately and measurably over time.

Beyond these two priorities, the project also leaves a reusable foundation behind it. The batch evaluation driver and the seven question test set mean that any future change, a new embedding model, a larger corpus, or a revised system prompt, can be measured against the same baseline reported here before it is adopted, turning what could easily have been a one-time class project into a small but genuine research instrument. That combination of a working application, an honest evaluation of its current limits, and a clear, evidence backed path to improving it is ultimately the most important deliverable of this work, more so than any single metric value reported in the tables above.

10. References

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). Ragas: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217. <https://arxiv.org/abs/2309.15217>

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 33. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>

Chroma. (2026). ChromaDB Documentation. <https://docs.trychroma.com/>

OpenAI. (2026). Vector Embeddings Guide, OpenAI API Documentation. <https://developers.openai.com/api/docs/guides/embeddings>

NASA. (2015). Apollo 11 Mission Overview. <https://www.nasa.gov/history/apollo-11-mission-overview/>

NASA. Apollo 13: Mission Details. <https://www.nasa.gov/missions/apollo/apollo-13-mission-details/>

Presidential Commission on the Space Shuttle Challenger Accident. (1986). Report to the President (Rogers Commission Report). <https://www.nasa.gov/history/rogersrep/genindex.htm>

Streamlit Inc. (2026). Streamlit Documentation. <https://docs.streamlit.io/>

LangChain. (2026). LangChain Documentation. <https://docs.langchain.com/>

Project source files: chat.py, embedding_pipeline.py, llm_client.py, rag_client.py, ragas_evaluator.py, batch_evaluate.py, test_questions.json, evaluation_results.json (NASA Mission Intelligence project repository).

Reference Verification

Every reference above was checked against a live, currently accessible source before inclusion in this report. Table 3 lists the verified destination for each entry.

Reference	Verified Source URL
Es et al., 2023 (RAGAS paper)	https://arxiv.org/abs/2309.15217
Lewis et al., 2020 (RAG paper)	https://arxiv.org/abs/2005.11401
ChromaDB Documentation	https://docs.trychroma.com/
OpenAI Vector Embeddings Guide	https://developers.openai.com/api/docs/guides/embeddings
NASA Apollo 11 Mission Overview	https://www.nasa.gov/history/apollo-11-mission-overview/
NASA Apollo 13 Mission Details	https://www.nasa.gov/missions/apollo/apollo-13-mission-details/
Rogers Commission Report, 1986	https://www.nasa.gov/history/rogersrep/genindex.htm
Streamlit Documentation	https://docs.streamlit.io/
LangChain Documentation	https://docs.langchain.com/

Table 3. Reference verification: each citation above was confirmed against a live, reachable source URL.